CLOUD COMPUTING

Assignment - 3

PARTIE 1 – CLOUDSIM 7.0.1

Exercise 1: Cost of cloudlets

On the basis of *CloudSim : Example2* ,

1. Copy the example into your package (e.g. *Ex1Costs*), run it unchanged and note the start and finish time of each cloudlet.
2. Modify it to simulate **3 cloudlets** of different lengths (for example 40,000, 80,000 and 120,000 MI).
3. The datacenter costs are defined in *createDatacenter()* (cost per second of CPU, per MB of memory, per MB of storage, per MB of bandwidth). Add a column **Cost** to the output, using *cloudlet.getProcessingCost()*.
4. Compute by hand the CPU cost of each cloudlet (actual CPU time × cost per second) and compare it with the value displayed. Explain any difference.
5. Calculate the total cost for the user. Which cloudlet is the most expensive, and why? How does the cost change if the cost per second is doubled?

Exercise 2: Heterogeneous hosts and execution time

On the basis of *CloudSim : Example3*, simulate **4 hosts** having each different characteristics and run **5 various cloudlets**. Suggested configuration:

Host	PEs	MIPS per PE	RAM (MB)	VM created on it
Host 0	1	1,000	2,048	VM 0: 1 PE, 1,000 MIPS
Host 1	2	1,500	4,096	VM 1: 1 PE, 1,500 MIPS
Host 2	4	2,000	8,192	VM 2: 2 PEs, 2,000 MIPS
Host 3	2	3,000	4,096	VM 3: 1 PE, 3,000 MIPS

1. Create 5 cloudlets of lengths 30,000, 60,000, 90,000, 120,000 and 150,000 MI, and assign each one to a VM with *cloudlet.setGuestId(vmId)*.
2. Show the execution time of each cloudlet and fill in a table: cloudlet, length, VM, VM MIPS, start time, finish time, execution time. Check that execution time ≈ length ÷ MIPS.
3. Explain the results. What is the **makespan** (finish time of the last cloudlet)?
4. How can we optimize the execution time? Propose a better assignment of cloudlets to VMs, simulate it and compare the makespan.

   Hint: assigning the longest cloudlets to the fastest VMs is the idea behind the LJF (Longest Job First) heuristic.

Exercise 3: Several datacenters and several users

On the basis of *CloudSim : Example4* (two datacenters) and *CloudSimExample5* (two users):

1. Create **3 datacenters** with **2 hosts** in each datacenter. Give each datacenter a different cost per second (e.g. 3.0, 2.0 and 1.0).
2. Create **3 users** (3 brokers), each with 2 VMs and 3 cloudlets. Do not forget to initialize CloudSim with the right number of users.
3. Execute the simulation and display, for each cloudlet, the datacenter and the VM used. Which datacenters actually receive VMs? Explain the results.
4. How can we optimize the execution time? How can we minimize the cost for each user? Are the two objectives compatible ?

Exercise 4: Time-shared vs space-shared scheduling

Scenario: 1 host with 1 PE of 1,000 MIPS, 1 VM with 1 PE, and 3 cloudlets of 10,000 MI each, submitted at the same time.

1. Predict on paper the start and finish time of each cloudlet with *CloudletSchedulerSpaceShared* and with *CloudletSchedulerTimeShared*. Draw a Gantt chart for each case.
2. Simulate both cases and compare with your predictions.
3. Now create 2 VMs of 1 PE on this host, first with *VmSchedulerSpaceShared*, then with *VmSchedulerTimeShared*. What happens to the second VM? Explain.
4. Which policy would you choose for an interactive web application? For batch HPC jobs? Justify.

PARTIE 2 – CLOUDSIM PLUS 8.5.7

Exercise 5: From CloudSim 7 to CloudSim Plus

1. Run *BasicFirstExample* and identify in the code the seven steps of a simulation seen in the course.
2. Rewrite **Exercise 2** with CloudSim Plus (same hosts, VMs and cloudlets) and display the results with *CloudletsTableBuilder*.
3. Compare the two versions: number of lines of code, results (are they identical?), readability. Which differences in the API do you notice?
4. Add costs to the datacenter (*getCharacteristics().setCostPerSecond(...)*, etc.) and display the total cost of each VM with *VmCost* (see *CostsExample1*). Compare with Exercise 1.

Exercise 6: VM allocation policies

Create one datacenter with 4 hosts (8, 4, 4 and 2 PEs of 1,000 MIPS) and 8 VMs requiring 1 to 4 PEs (choose the sizes so that they can all fit).

1. Run the simulation with each of the following policies, passed to the *DatacenterSimple* constructor: *VmAllocationPolicySimple* (default), *VmAllocationPolicyFirstFit*, *VmAllocationPolicyBestFit*, *VmAllocationPolicyRoundRobin*.
2. Fill in a table: policy, number of hosts used, VMs placed on each host, VMs that could not be created, makespan.
3. Which policy is best to save energy (consolidation)? Which is best to spread the load?

Exercise 7: Brokers and load balancing

1. Do the examples of CloudSim 7 and CloudSim Plus include load-balancing and/or broker algorithms? Browse the examples, give the class names and explain how each algorithm works.
2. Create 3 VMs of 1,000, 2,000 and 4,000 MIPS and 12 cloudlets of random lengths (between 10,000 and 100,000 MI, fixed seed). Compare the makespan (total execution time) obtained with *DatacenterBrokerSimple* and with *DatacenterBrokerBestFit*.